

ASSIGNMENT DETAILS

Unit Code	COS10009	Unit Title	Introduction to Programming
Tutorial/Lab Group	Lab Group 2	Lecturer/Tutor Name	Dr Jason Yong & Miss Carmen Wang Er Chai
Assignment Title	Custom Program Design Report		
Due date	3rd December 2023	Date Received	

DECLARATION

For both individual and group assignments, in the case of assignment submission on behalf of another student, it is assumed that permission has been given. The University takes no responsibility for any loss, damage, theft, or alteration of the assignment.

To be completed if this is an individual assignment

I declare that this assignment is my individual work. I have not worked collaboratively, nor have I copied from any other student's work or from any other source/s, except where due acknowledgment is made explicitly in the text, nor has any part been written for me by another person.

Student Details	Student ID Number	Student Name	Student Signature
Student 1	102787457	Christine Khai Ning BONG	

To be completed if this is a group assignment

We declare that this is a group assignment and that no part of this submission has been copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part been written for us by another person.

Student Details	Student ID Number(s)	Student Name(s)	Student Signature (s)
Student 1			
Student 2			
Student 3			
Student 4			
Student 5			

MARKER'S COMMENTS

Total Mark	Marker's Signature	Date
------------	--------------------	------

EXTENSION CERTIFICATE

This assignment has been given an extension by

Unit Convenor	Date Received
Extended due date	

COS10009 Introduction to Programming
Semester 2, 2023
Custom Program
Design Report

Name: Christine Khai Ning BONG
Student ID: 102787457

Table of Content

1. Flowchart -----	3
2. Function Prototypes & Structure Definition -----	4
3. Main Structure -----	5, 6
4. Add Event -----	6, 7
5. Event Records -----	8
6. Search Event -----	9
7. Modify Event -----	10, 11
8. Delete Event -----	12, 13
9. myExit -----	14
10.Improvement -----	15
11.References -----	15

1. Flowchart

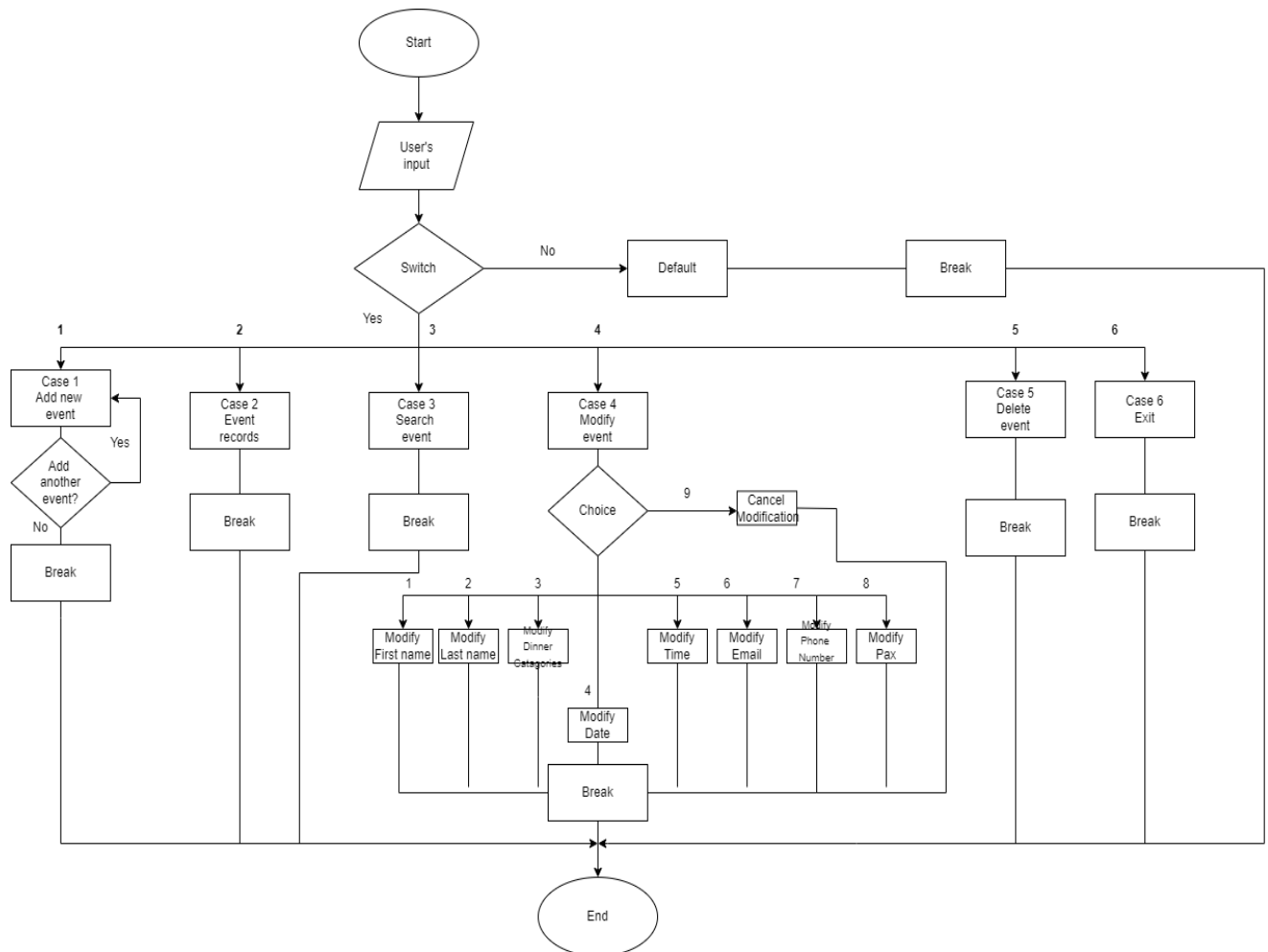


Figure 1. Flowchart of the event management

2. Function Prototypes & Structure Definition

```
1  #include <stdio.h>
2  #include <conio.h>
3  #include <stdlib.h>
4  #include <string.h>
5
6  void addevent();
7  void eventrecords();
8  void searchevent();
9  void modifyevent();
10 void deleteevent();
11 void myExit();
12
13 typedef struct information{
14     char first_name[64];
15     char last_name[64];
16     char categories[64];
17     int year;
18     int month;
19     int day;
20     char time[10];
21     char email[50];
22     int phone_number;
23     int pax;
24 } event;
```

Figure 2. Coding for Function Prototypes & Structure Definition

In this software, there are 6 void functions, which are Add Event, Event Records, Search Event, Modify Event, Delete Event, myExit, will be explained in this report. Typedef struct here defines a few basic information which needed to get from users as event's information. For printing out the date and time, <string.h> library was purposely included for the time as it was set as a string; for the date, instead of "int date", save users input as "int year", "int month", "int day" and with enough command, it will be printed out more neatly.

3. Main structure

```

3 > typedef struct information{ ...
24 } event;
25
26 void main(){
27     int choice = 0;
28     while(choice != 6){
29         printf("\t\t\t\t\t===EVENT MANAGEMENT===\t\t\t\t\t");
30         printf("\n\n\t\t\t\t\t1. Add new event\n");
31         printf("\n\t\t\t\t\t2. Event records\n");
32         printf("\n\t\t\t\t\t3. Search event\n");
33         printf("\n\t\t\t\t\t4. Modify event\n");
34         printf("\n\t\t\t\t\t5. Delete event\n");
35         printf("\n\t\t\t\t\t6. Exit\n");
36         printf("\t\t\t\t\t_____ \n");
37         printf("\n\t\t\t\t\t ");
38         scanf("%d", &choice);
39
40         switch(choice){
41             case 1:
42                 system("cls");
43                 addevent();
44                 system("cls");
45                 break;
46
47             case 2:
48                 system("cls");
49                 eventrecords();
50                 printf("\t\t\t\t\t Press any key to exit.\n");
51                 getch();

```

```

C:\event_management> @ searchevent()
50         printf("\t\t\t\t\t Press any key to exit.\n");
51         getch();
52         system("cls");
53         break;
54
55         case 3:
56             system("cls");
57             searchevent();
58             printf("\t\t\t\t\t Press any key to exit.\n");
59             getch();
60             system("cls");
61             break;
62
63         case 4:
64             system("cls");
65             modifyevent();
66             printf("\t\t\t\t\t Press any key to exit.\n");
67             getch();
68             system("cls");
69             break;
70
71         case 5:
72             system("cls");
73             deleteevent();
74             printf("\t\t\t\t\t Press any key to exit.\n");
75             getch();
76             system("cls");
77             break;
78
79         case 6:

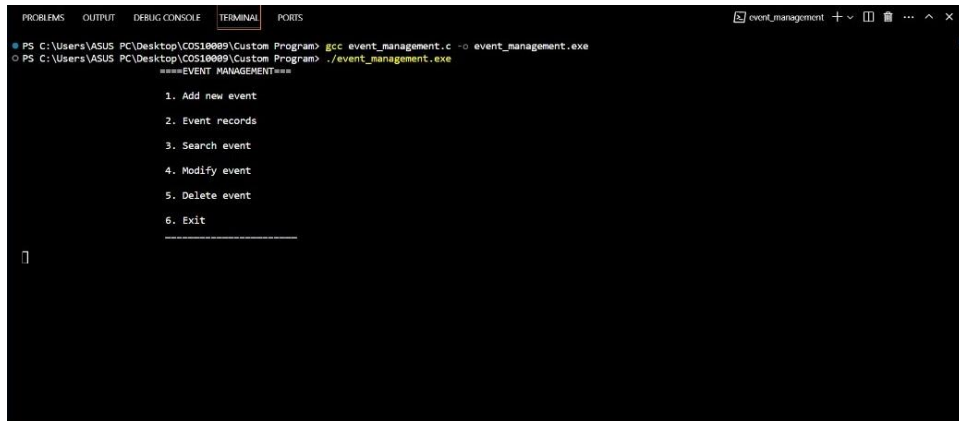
```

```

C:\event_management> @ searchevent()
70
71         case 5:
72             system("cls");
73             deleteevent();
74             printf("\t\t\t\t\t Press any key to exit.\n");
75             getch();
76             system("cls");
77             break;
78
79         case 6:
80             system("cls");
81             printf("\n\t\t\t\t\t Thank you for using this software.\n\n");
82             exit(0);
83             break;
84
85         default:
86             system("cls");
87             getch();
88             printf("\n\t\t\t\t\t Enter a valid number");
89             printf("\n\t\t\t\t\t Press any key to continue.\n");
90             getch();
91             system("cls");
92             break;
93     }
94 }
95
96 }
97
98 }

```

Figure 3, 4, 5. Coding for Main Structure



```
PS C:\Users\ASUS PC\Desktop\COS10009\Custom Program> gcc event_management.c -o event_management.exe
PS C:\Users\ASUS PC\Desktop\COS10009\Custom Program> ./event_management.exe

====EVENT MANAGEMENT====

1. Add new event
2. Event records
3. Search event
4. Modify event
5. Delete event
6. Exit
```

Figure 6. Terminal for Main Structure

Here is the main menu that users will be able to see. So, choices from 1 to 5 will bring them to different functions, and choice number 6 will exit the software. For integer choice it is equal to 0 to initialize it, and the while loop is for when choices are equal from 1 to 5, it can run the program, and equals to 6 it can exit. For the functions, they are in switch cases as easy to perform actions than “if-else”, also I have put “system(“cls”)” to clear the console screen. Of course, there is also a default case to remind users to type in a valid number.

4. Add Event

```
99
100 void addevent(){
101     char another;
102     FILE *fptr;
103     event info;
104
105     do{
106         system("cls");
107         printf("\t\t\t\t\t === Add new event ===\n\n");
108         fptr=fopen("event_info.txt","a+");
109         if (fptr == NULL){
110             fprintf(stderr, "\t\t\t\t\t Cannot open file.");
111         } else {
112
113             printf("\t\t\t\t\t First name: ");
114             scanf("%s", info.first_name);
115             printf("\t\t\t\t\t Last name: ");
116             scanf("%s", info.last_name);
117             printf("\t\t\t\t\t Dinner catogories (Wedding/Birthday/Year-end): ");
118             scanf("%s", info.catogories);
119             printf("\t\t\t\t\t Date(yyyy/mm/dd): ");
120             scanf("%d/%d/%d", &info.year, &info.month, &info.day);
121
122             while (getchar() != '\n');
123
124             printf("\t\t\t\t\t Time (hh:mm): ");
125             fgets(info.time, sizeof(info.time), stdin);
126             info.time[strcspn(info.time, "\n")] = '\0'; // Remove newline character from the end
127             printf("\t\t\t\t\t Email: ");
```

```

118     scanf("%s", info.categories);
119     printf("\t\t\t\t\t Date(yyyy/mm/dd): ");
120     scanf("%d/%d/%d", &info.year, &info.month, &info.day);
121
122     while (getchar() != '\n');
123
124     printf("\t\t\t\t\t Time (hh:mm): ");
125     fgets(info.time, sizeof(info.time), stdin);
126     info.time[strcspn(info.time, "\n")] = '\0'; // Remove newline character from the end
127     printf("\t\t\t\t\t Email: ");
128     scanf("%s", info.email);
129     printf("\t\t\t\t\t Phone number (+60): ");
130     scanf("%d", &info.phone_number);
131     printf("\t\t\t\t\t Pax: ");
132     scanf("%d", &info.pax);
133     printf("\n\t\t\t\t\t _____\n");
134
135     fwrite(&info, sizeof(event), 1, fptr);
136     printf("\t\t\t\t\t Information stored successfully.");
137
138     fclose(fptr);
139 }
140
141 printf("\t\t\t\t\t Do you want to add another record?(Y/N)");
142 getchar();
143 scanf(" %c", &another);
144
145 } while (another == 'y' || another == 'Y');
146
147 }

```

Figure 7.8. Coding for Add Event

```

===== Add new event =====

First name: Cindy
Last name: Wang
Dinner catogories (Wedding/Birthday/Year-end): Birthday
Date(yyyy/mm/dd): 2024/01/03
Time (hh:mm): 20:00
Email: cindy549@gmail.com
Phone number (+60): 1157869
Pax: 20

Information stored succesfully.

Do you want to add another record?(Y/N)

```

Figure 9. Terminal for Add Event

This is the first function, add event, in the software. Open the file “event_info.txt” at the beginning to write and read from the file. For the date, it must be saving as the integer separately as it will show neatly, and for the time, it will skip it directly and jump to the next request, so must be done manually, to get character then only move to next request, here “strcspn” is used to find the initial of the string. Plus, an if-else inside to determine whether the text document can be open or not, also a do-while loop outside the add event function coding, as users want to add another event, so when we get a “yes” from them then the whole function can run again.

5. Event Records

```

149 void eventrecords(){}
150 FILE *fptr;
151 event info;
152 fptr = fopen("event_info.txt", "r");
153
154 printf("\t\t\t\t\t === Event Records ===\n\n\n");
155
156 if(fptr == NULL){
157     fprintf(stderr, "\t\t\t\t\tCannot open file.");
158 } else {
159     printf("\t\t\t\t\tInformation\n");
160     printf("\t\t\t\t\t_____ \n\n");
161 }
162
163 int i = 1; //Initialize i outside the loop
164 while(fread(&info, sizeof(event), 1 , fptr) == 1){
165     printf("\n\t\t\tRecord %d:", i);
166     printf("\n\t\t\tName           : %s %s", info.first_name, info.last_name);
167     printf("\n\t\t\tDinner categories   : %s", info.categories);
168     printf("\n\t\t\tDate             : %d/%d/%d", info.year, info.month, info.day);
169     printf("\n\t\t\tTime              : %s", info.time);
170     printf("\n\t\t\tEmail            : %s", info.email);
171     printf("\n\t\t\tPhone number (+60) : %d", info.phone_number);
172     printf("\n\t\t\tPax               : %d", info.pax);
173     printf("\n\t\t\t\t\t_____ \n\n");
174     i++;
175 }
176 fclose(fptr);
177 getch();

```

Figure 10. Coding for Event Records



Figure 11. Terminal for Event Records

For the second function in the software, the file pointer is opened to read the data in the “event_info.txt” text document. An if-else to determine whether the text document can be opened or not. While loop for while it can be opened, then print the information data, and to show the sequence of the record I put an integer “i” as a counter outside the loop and equal to 1 to check if it has been read.

6. Search Event

```

182 void searchEvent(){
183     FILE *fptr;
184     event info;
185     int phone_number, found = 0;
186     fptr = fopen("event_info.txt", "r");
187
188     printf("\t\t\t\t\t==== Search event ===\n\n\n");
189     printf("\t\t\t\t\tEnter phone number: ");
190     scanf("%d", &phone_number);
191
192     while(fread(&info, sizeof(event), 1, fptr)){
193         if(info.phone_number == phone_number){
194             found = 1;
195             printf("\n\t\t\t\t\tName           : %s %s", info.first_name, info.last_name);
196             printf("\n\t\t\t\t\tDinner categories : %s", info.categories);
197             printf("\n\t\t\t\t\tDate            : %d/%d/%d", info.year, info.month, info.day);
198             printf("\n\t\t\t\t\tTime            : %s", info.time);
199             printf("\n\t\t\t\t\tEmail           : %s", info.email);
200             printf("\n\t\t\t\t\tPhone number    (+60) : %d", info.phone_number);
201             printf("\n\t\t\t\t\tPax             : %d", info.pax);
202             printf("\n\t\t\t\t\t_____ : %d", info.pax);
203             printf("\n\t\t\t\t\t_____ \n");
204         }
205         if(!found){
206             printf("\t\t\t\t\tRecord not found.\n");
207         }
208         fclose(fptr);
209         getch();
210     }
}

```

Figure 12. Coding for Search Event



The screenshot shows a terminal window with a dark background. The title bar at the top includes tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The terminal content shows a command prompt '==== Search event ====' followed by the input 'Enter phone number: 1157869'. The output is a list of details for a dinner event: Name: Cindy Wang, Dinner categories: Birthday, Date: 2024/1/3, Time: 20:00, Email: cindys49@gmail.com, Phone number (+60): 1157869, and Pax: 20. A dashed line separates the output from the rest of the terminal.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
==== Search event ====
Enter phone number: 1157869

Name           : Cindy Wang
Dinner categories : Birthday
Date           : 2024/1/3
Time           : 20:00
Email          : cindys49@gmail.com
Phone number (+60) : 1157869
Pax            : 20
-----
```

Figure 13. Terminal for Search Event

For the third function, search event, the file pointer was opened to read data from the file. In here, phone number is used to search for the matched data, if it is found, it will run the while loop and if it is not, it will tell us "Record not found.". "Found = 0" is to initialize the variable "found", to see does the search runs successfully.

7. Modify Event

```

212 void modifyevent(){
213     FILE *fptr, *fptrl;
214     event info;
215     int phone_number, found = 0;
216
217     printf("\t\t\t\t===MODIFY EVENT=== \n\n");
218     fptr = fopen("event_info.txt", "r");
219     fptrl = fopen("temp.txt", "w+");
220
221     if (fptr == NULL){
222         fprintf(stderr, "\t\t\t\tCannont open file.");
223         return;
224     }
225
226     printf("\t\t\t\tEnter phone number to search: ");
227     scanf("%d", &phone_number);
228     int choice = 0; // Initialize choice before the loop
229     while (fread(&info, sizeof(event), 1, fptr)){
230         if (info.phone_number == phone_number){
231             found = 1;
232
233             printf("\t\t\t\t===CURRENT INFORMATION===\n");
234             printf("\t\t\t\t[1] First Name      : %s", info.first_name);
235             printf("\t\t\t\t[2] Last Name       : %s", info.last_name);
236             printf("\t\t\t\t[3] Dinner categories : %s", info.categories);
237             printf("\t\t\t\t[4] Date           : %d/%d/%d", info.year, info.month, info.day);
238             printf("\t\t\t\t[5] Time            : %s", info.time);
239
240             printf("\t\t\t\t[6] Pax              : %d", info.pax);
241             printf("\t\t\t\t[9] Cancel modification");
242             printf("\t\t\t\tEnter your choice: ");
243             scanf("%d", &choice);
244
245             switch(choice){
246                 case 1:
247                     printf("\t\t\t\tEnter new First Name: ");
248                     scanf("%s", info.first_name);
249                     break;
250                 case 2:
251                     printf("\t\t\t\tEnter new Last Name: ");
252                     scanf("%s", info.last_name);
253                     break;
254                 case 3:
255                     printf("\t\t\t\tEnter new Categories: ");
256                     scanf("%s", info.categories);
257                     break;
258                 case 4:
259                     printf("\t\t\t\tEnter new Date(yyyy/mm/dd): ");
260                     scanf("%d/%d/%d", &info.year, &info.month, &info.day);
261                     break;
262                 case 5:
263                     printf("\t\t\t\tEnter new Time: ");
264                     scanf("%s", info.time);
265                     break;
266                 case 6:
267                     printf("\t\t\t\tEnter new Email: ");
268                     scanf("%s", info.email);
269                     break;
270                 case 9:
271                     break;
272                 default:
273                     printf("\t\t\t\tInvalid choice. Please enter a valid number.\n");
274                     break;
275             }
276
277             //Write modified information to new file
278             fwrite(&info, sizeof(event), 1, fptrl);
279         } else {
280             //Copy unchanged records to the new file
281             fwrite(&info, sizeof(event), 1, fptrl);
282         }
283     }
284     fclose(fptr);
285     fclose(fptrl);
286
287     if (found && choice != 9){
288         //Remove the old file and rename the new file
289         remove("event_info.txt");
290         rename("temp.txt", "event_info.txt");
291         printf("\t\t\t\tRecord modified successfully.\n");
292     } else if (choice == 9){
293         printf("\t\t\t\tModification canceled.\n");
294     } else {
295         printf("\t\t\t\tRecord not found.\n");
296     }
297     getch();
298 }

```

Figure 14, 15, 16. Coding for Modify Event

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
===MODIFY EVENT===

Enter phone number to search: 1157869
===CURRENT INFORMATION===

[1] First Name      : Cindy
[2] Last Name       : Wang
[3] Dinner categories : Birthday
[4] Date            : 2024/1/3
[5] Time            : 20:00
[6] Email           : cindy549@gmail.com
[7] Phone number (+60) : 1157869
[8] Fax             : 20
[9] Cancel modification

Enter your choice: 7

Enter new Phone Number: 1157869532

Record modified successfully.
```

A screenshot of a terminal window with a dark background. The terminal has tabs at the top labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. In the top right corner of the terminal, there is a toolbar with icons for search, expand, close, and other window management functions. The text in the terminal shows a search event for a birthday. It starts with '==== Search event ===', followed by 'Enter phone number: 1157869532'. Then, a list of details is shown: 'Name : Cindy Wang', 'Dinner categories : Birthday', 'Date : 2024/1/3', 'Time : 20:00', 'Email : cindy549@gmail.com', 'Phone number (+60) : 1157869532', and 'Pax : 20'. A horizontal line is drawn after the 'Pax' line. The cursor is at the end of the line.

Figure 17,18 Terminal for Modify Event and Search Event

This is the fourth function in the software, modify event's data, in this function users also need to enter their phone number to search for their record. Two file pointers were opened; one is for the original file (event_info.txt), and one is for the file (temp.txt) to save the modified data and rename it as the original file. From case 1 to 8 is to let users choose which data they want to modify, and case 9 is to cancel modification, and for this there is also an if for either phone number is found, and what action should be taken and what messages should be printed.

8. Delete Event

```

311 void deleteevent(){
312     FILE *fptr, *fptr1;
313     event info;
314     int phone_number, found = 0;
315
316     printf("\t\t\t\t\t=== Delete event ===\n\n\n");
317     fptr = fopen("event_info.txt", "r");
318     fptr1 = fopen("temp.txt", "w");
319     printf("\t\t\t\t\tEnter phone number: ");
320     scanf("%d", &phone_number);
321
322     if(fptr == NULL){
323         fprintf(stderr, "\t\t\t\t\tCannot open file.");
324     }
325     while(fread(&info, sizeof(event), 1, fptr)){
326         if(info.phone_number == phone_number){
327             found = 1;
328         } else {
329             fwrite(&info, sizeof(event), 1, fptr1);
330         }
331     }
332     fclose(fptr);
333     fclose(fptr1);
334
335     if(found){
336         remove("event_info.txt");
337         rename("temp.txt", "event_info.txt");
338         printf("\n\t\t\t\t\tRecord deleted successfully.\n\n");
339     }
340 }

```

```

319     fptr1 = fopen("temp.txt", "w");
320     printf("\n\t\t\tEnter phone number:");
321     scanf("%d", &phone_number);
322
323     if(fptr == NULL){
324         fprintf(stderr, "\n\t\t\tCannot open file.");
325     }
326     while(fread(&info, sizeof(event), 1, fptr)){
327         if(info.phone_number == phone_number){
328             found = 1;
329         } else {
330             fwrite(&info, sizeof(event), 1, fptr1);
331         }
332     }
333     fclose(fptr);
334     fclose(fptr1);
335
336     if(found){
337         remove("event_info.txt");
338         rename("temp.txt", "event_info.txt");
339         printf("\n\t\t\tRecord deleted successfully.\n");
340     }
341
342     if(!found){
343         printf("\n\t\t\tRecord not found.\n");
344     }
345     getch();
346 }
347

```

Figure 19, 20. Coding for Delete Event

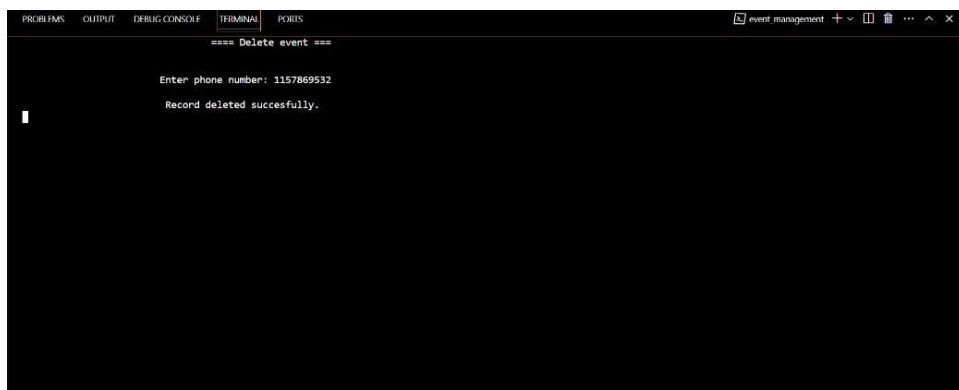




Figure 21, 22. Terminal for Delete Event and Event Records

The fifth function in the software, delete event's data, in this function users need to enter their phone number to search for their record. Two file pointers were also opened here; one is for the original file (event_info.txt), and one is for the file (temp.txt). So, when data is deleted in the original file, the original file will be removed, and the "temp.txt" file will be renamed to "event_info.txt" which is the original file's name with the remaining data. There are also two "if" for either record is found or not with its related action and messages.

9. myExit

```
182 > void searchevent(){...
211
212 > void modifyevent(){...
310
311 > void deleteevent(){...
348
349 void myExit(){
350
351     printf("\n\t\t\t Thank you for using this software.\n\n");
352     exit(0);
353
354 }
```

Figure 23. Coding for myExit

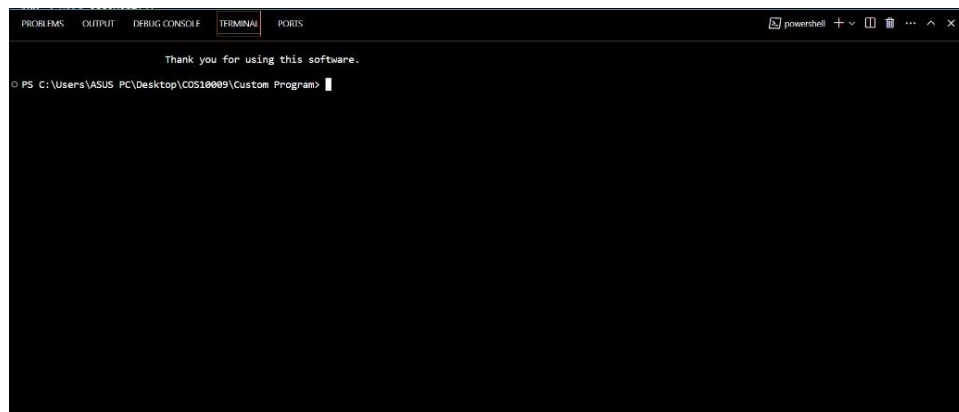


Figure 24. Terminal for myExit

The last function in this software, the myExit, was named “Exit” before this, since it would crash with the standard library function, so it was changed to “myExit”. In this case, “exit(0)” is to call the standard exit function, so when users choose 6, it will exit the software with a thank you message.

10. Improvement

In this software, I still see there are a few improvements that can be made.

1. The do-while loop for Search, Modify and Delete Event function, ask users is there another record that they still want to make changes with or not.
2. User's login with Username and Password at the very beginning of the software.
3. Develop this software for two kinds of users, which is event manager (admin account) and their customers (users account; some functions might be deducted). This could be easily differentiated by the user's login.
4. For admin account, they will be able to see the upcoming and past events, which `<time.h>` library need to be included.

11. References

1. YouTube video 1

<https://youtu.be/E9HHKiwwlH8?si=6S-4gjOC2yLOEzzy>

2. YouTube video 2

<https://youtu.be/CVI68c8ki34?si=WFSsjL63hMMkV3Bi>

3. Sources of inspiration

<https://youtu.be/6rjG2ld9qAc?si=KSqeLHoF2o9LLajG>

4. How to obtain a time in C using a struct

<https://stackoverflow.com/questions/44768107/c-how-to-obtain-a-time-in-c-using-a-struct>